

quantum-safe: Bridging the Post-Quantum Production Gap with a Hybrid-by-Default Python Cryptography Library

Animesh Shaw, *Independent Researcher*

Abstract—The August 2024 finalisation of FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) closed the algorithmic gap in post-quantum cryptography (PQC). The production gap — hybrid combiners, versioned key formats, protocol helpers, and migration tooling — remains open. We present `quantum-safe`, a Python library that closes all three critical gaps we identify, and a systematic evaluation of the nine-library ecosystem that quantifies them.

We score nine PQC libraries across eight production-readiness dimensions. Three dimensions have coverage below 35%: hybrid KEM support (11%), migration tooling (22%), and protocol integration (33%). `quantum-safe` scores Full on all eight. The full API reduces the hybrid KEM task from 45 lines of manual combiner code to three lines, directly lowering the risk of insecure combiner implementations.

We report the first statistically rigorous per-operation overhead measurement for a Python hybrid PQC library (3,000 iterations, CPU-pinned, bootstrapped 95% confidence intervals). A full X25519+ML-KEM-768 handshake completes in 243 μ s under Docker/Linux — 0.5–2.5% of a typical TLS 1.3 round-trip budget. At 5,000 concurrent users, throughput holds at 2,848 ops/s with only 4.9% degradation versus the single-user baseline, confirming that `liboqs` releases the Python GIL during C-level operations.

We introduce Coefficient of Variation (CoV) as a practical timing side-channel proxy across all FIPS 203/204 operations. ML-KEM-768 decapsulation achieves CoV = 3.9%, within the AES-256-GCM noise floor (2.1%). ML-DSA-65 signing shows CoV = 51.5%, expected from FIPS 204 rejection sampling, not a side-channel. This CoV methodology has not previously been applied to PQC library evaluation and provides a lightweight complement to formal constant-time verification tools. All results are reproducible via a single Docker command.

Index Terms—post-quantum cryptography, hybrid key encapsulation, ML-KEM, ML-DSA, FIPS 203, FIPS 204, Python cryptography library, timing side-channel, Coefficient of Variation, TLS 1.3, production readiness, HKDF

I. INTRODUCTION

ON 13 August 2024, the National Institute of Standards and Technology (NIST) published three post-quantum cryptography standards: FIPS 203 [1], FIPS 204 [2], and FIPS 205 [3]. These documents standardise ML-KEM (derived from CRYSTALS-Kyber [4]), ML-DSA (derived from CRYSTALS-Dilithium [5]), and SLH-DSA as the drop-in replacements for RSA and ECDH in key exchange and digital signatures.

The algorithmic question is settled. The production question is not.

Manuscript received May 19, 2026. A. Shaw is an independent security researcher. E-mail: animesh15b@iimk.edu.in.

The `quantum-safe` library, 415 tests, and all benchmark data are available at <https://github.com/AnimeshShaw/quantum-safe> (to be made public upon acceptance). All benchmarks are fully reproducible via Docker; see Section V.

Consider what a software engineer must do today to add hybrid post-quantum key exchange to a Python service. They must install `liboqs-python`, write an X25519 key generation step, write a separate ML-KEM key generation step, concatenate the two public keys, run HKDF to combine the two shared secrets, wrap the result in a key derivation function appropriate for their protocol, and handle serialisation for both halves. The result is roughly 45 lines of boilerplate before the first application-specific byte is encrypted. Then they repeat this exercise for signatures, certificates, and protocol configuration.

This friction is not accidental. It reflects a genuine gap in the PQC ecosystem: algorithm implementations exist, but the production layer — hybrid combiners, versioned key formats, protocol helpers, migration tooling — does not. Insecure combiner implementations are a known risk [6], and prior work on cryptographic API usability shows that complexity directly predicts misuse [7], [8].

A. The Harvest Now, Decrypt Later Threat

The urgency is real. Adversaries with access to encrypted network traffic can store ciphertext today and decrypt it once a cryptographically relevant quantum computer becomes available [9]. This “harvest now, decrypt later” (HNDL) attack is passive, undetectable, and already underway. Data with long confidentiality requirements — government communications, financial records, healthcare data — is already at risk under purely classical encryption.

The NSA’s Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) [10] mandates algorithm replacement by 2030 for national security systems. Commercial organisations face comparable regulatory pressure. The gap between the 2024 standard finalisation and the 2030 deadline is not large given the complexity of cryptographic migration in production infrastructure.

B. The Production Gap

Despite this urgency, deployment stalls at the library layer. To understand why, we evaluated nine actively maintained PQC libraries across eight dimensions measuring production readiness. The results are shown in Fig. 1. Three dimensions have coverage below 35%:

- **Hybrid KEM support (11%):** Only `cloudflare/circl` provides a built-in hybrid combiner. Every other library exposes raw algorithm primitives, placing the burden of combining X25519 with ML-KEM on the application developer.
- **Migration path (22%):** No library except `Bouncy Castle` (partially) provides tooling for upgrading classical

keypairs, re-encrypting stored ciphertext, or scanning codebases for classical cryptography imports.

- **Protocol integration (33%):** Application developers need TLS configuration helpers, X.509 certificate generation, and envelope encryption. Only three libraries provide partial protocol support; none provides full coverage.

C. Companion Work

A companion paper [11] presents the Quantum-Safe Auditor (QSA), a static analysis tool that detects classical cryptography usage in Python codebases and recommends migration paths. QSA identifies *where* PQC migration is needed; *quantum-safe* provides the *implementation* to migrate to. The two tools are designed to work together in a production migration workflow.

D. Contributions

This paper makes six contributions:

- 1) **Production gap matrix.** A systematic, reproducible evaluation of nine PQC libraries across eight production-readiness dimensions using a ternary rubric.
- 2) **Hybrid overhead quantification.** The first statistically rigorous per-operation latency measurement for a Python hybrid PQC library, including bootstrap 95% confidence intervals, Welch's *t*-test, and Cohen's *d* effect sizes. Full handshake: 243 μ s (Docker/Linux).
- 3) **Concurrent throughput analysis.** The first published throughput-vs-concurrency curve for a Python hybrid PQC library, from 100 to 5,000 simultaneous users. Throughput degrades only 4.9% across a 50 $\times$ increase in concurrent load.
- 4) **CoV as timing side-channel proxy.** A systematic Coefficient of Variation analysis across all FIPS 203/204 operations, using AES-256-GCM as a constant-time noise floor reference.
- 5) **Zero-config hybrid API.** The *quantum-safe* library reduces the hybrid KEM task from 45 lines of manual combiner code to three lines.
- 6) **Open-source artefact.** All code, 415 tests, and the complete benchmark harness are released under the Apache 2.0 licence.

E. Paper Organisation

Section II covers the cryptographic background. Section III presents the gap analysis. Section IV describes library design. Section V presents benchmark methodology. Section VI presents results. Section VII discusses limitations and deployment implications. Section VIII surveys related work. Section IX concludes.

II. BACKGROUND

A. Post-Quantum Cryptography Standards

ML-KEM [1] is a key encapsulation mechanism based on the module learning-with-errors (MLWE) problem. It replaces Diffie-Hellman and ECDH in key exchange. Three

parameter sets are defined: ML-KEM-512 (NIST security level 1, AES-128 equivalent), ML-KEM-768 (level 3, AES-192 equivalent), and ML-KEM-1024 (level 5, AES-256 equivalent). *quantum-safe* defaults to ML-KEM-768, balancing security margin with performance. The MLWE problem is believed to be hard even for polynomial-time quantum algorithms, unlike the discrete logarithm and integer factorisation problems underpinning RSA and ECDH.

ML-DSA [2] is a digital signature scheme also built on MLWE. Its signing algorithm uses randomised hedged signing with a rejection-sampling loop, which causes variable execution time. This is a deliberate security design: the variable time is independent of the signing key, preventing fault-injection attacks that exploit deterministic signing. We discuss the timing implications in Section VI-D.

SLH-DSA [3] is a stateless hash-based signature scheme offering security from different mathematical assumptions than ML-KEM and ML-DSA. It is not benchmarked here as *quantum-safe* does not yet implement SLH-DSA; it is planned for a future release. Hash-based schemes are important as a fallback if lattice-based assumptions are weakened.

B. Hybrid Key Exchange

The rationale for hybrid construction is defence in depth: if a quantum computer breaks ML-KEM but classical ECDH remains secure (or vice versa), the hybrid secret remains secure. Formally, given independent shared secrets ss_{X25519} and ss_{ML-KEM} , the combined shared secret:

$$ss = \text{HKDF-SHA256}(ss_{X25519} \parallel ss_{ML-KEM}, \text{salt}, \text{info}) \quad (1)$$

is secure as long as at least one component shared secret is computationally hidden from the adversary [6]. This composability property — the hybrid is at least as secure as its strongest component — is the formal justification for deploying hybrid KEM during the transition period.

CNSA 2.0 [10] requires hybrid operation through 2030. Cloudflare enabled X25519+ML-KEM hybrid TLS for all its servers in 2022 [12], and Google Chrome deployed X25519MLKEM768 in 2023 [13].

C. Timing Side Channels

Timing side-channel attacks exploit correlations between secret values and execution time. Kocher's 1996 attack on RSA implementations [14] demonstrated that timing differences of nanoseconds can be amplified over many observations to recover private keys. Remote timing attacks on TLS implementations remained practical as recently as 2011 [15].

For lattice-based schemes, the primary constant-time requirement is that decapsulation and decryption do not branch on secret data. ML-KEM specifies this explicitly in FIPS 203 Section 6. Formal constant-time verification uses tools like *ct-verif* [16] or *dudect* [17], which operate at the binary or hardware level. Our CoV analysis provides a practical first-order screen applicable at the Python library level, where formal tools are not directly applicable.

D. The Python GIL and Cryptographic Concurrency

CPython’s Global Interpreter Lock (GIL) serialises Python bytecode execution, preventing true thread parallelism for pure-Python code [18]. However, C extension modules can release the GIL during computationally intensive operations. `liboqs` [19] is a C library; whether its Python binding releases the GIL has not previously been measured empirically. Our concurrent load tests provide the first published evidence.

Note that Python 3.13 introduced optional GIL disabling (PEP 703). The measurements in this paper are from Python 3.12 with the GIL enabled, which remains the dominant deployment target. The behaviour under free-threaded Python 3.13+ is left for future work.

E. Key Serialisation and Versioning

Post-quantum public keys are substantially larger than their classical counterparts: an ML-KEM-768 public key is 1,184 bytes versus 32 bytes for X25519. A hybrid public key must carry both components together with an algorithm identifier, version field, and parameter set. Without a standard format, each application invents its own, preventing interoperability and complicating future migration. The `quantum-safe` library addresses this with a CBOR-based [20] versioned key format described in Section IV-D.

III. THE PQC PRODUCTION GAP

A. Evaluation Methodology

We evaluated nine PQC libraries that are either (a) the primary PQC binding in their language ecosystem, (b) the implementation recommended in NIST migration guidance, or (c) actively maintained with post-FIPS-203/204 support. Table I lists the libraries.

TABLE I
PQC LIBRARIES EVALUATED (STATE AS OF MARCH 2026)

#	Library	Language	Notes
1	<code>liboqs-python</code>	Python	OQS project Python binding
2	<code>cryptography (pyca)</code>	Python	De-facto Python crypto library
3	<code>liboqs-js</code>	JavaScript	OQS project JS binding
4	<code>noble-post-quantum</code>	JavaScript	Audited pure-JS PQC
5	<code>pqcrypto</code>	Rust	FFI wrappers
6	<code>RustCrypto ml-*</code>	Rust	Pure Rust, trait-based
7	<code>oqs (Rust)</code>	Rust	OQS project Rust crate
8	<code>cloudflare/circl</code>	Go	Production-deployed
9	<code>Bouncy Castle</code>	Java	Enterprise Java crypto

Each library was assessed against eight production-readiness dimensions using a ternary scoring rubric: Full (F), Partial (P), or None (N). Scores were assigned by inspecting library documentation, source code, published API references, and PyPI/npm/crates.io package metadata as of March 2026. The scoring rubric for each dimension is defined in Table II.

B. Results

Fig. 1 shows the complete matrix. Table III summarises ecosystem coverage per dimension.

TABLE II
SCORING RUBRIC — EIGHT PRODUCTION-READINESS DIMENSIONS

Dimension	Full (F) requires
Unified API	Single import covers KEM + Sign + key management
Hybrid KEM	Built-in X25519 + ML-KEM combiner; one API call
FIPS 203/4/5	All three final standards implemented and exposed
Algo Agility	Algorithm selection without code change (registry or trait)
WASM	Published browser/edge deployable package
Ready Dev / CI	Typed API, comprehensive tests, active CI pipeline
Migration Path	Versioned key formats, upgrade tooling, classical scanner
Protocol Layer	At least two of: TLS config, X.509, JWT, envelope encryption

TABLE III
PQC ECOSYSTEM COVERAGE BY DIMENSION (9 LIBRARIES)

Dimension	Full	Partial	Coverage (F+P)
Algo Agility	4	5	100%
FIPS 203/4/5	5	3	89%
Unified API	2	7	100%
Dev / CI	5	4	100%
WASM Ready	3	1	44%
Protocol Layer	0	3	33%
Migration Path	0	2	22%
Hybrid KEM	1	1	22%

C. Critical Gaps and Per-Library Analysis

Hybrid KEM (11% Full, 22% with Partial). Only `cloudflare/circl` (Go) provides a production-ready hybrid combiner. `Bouncy Castle` offers partial support via its composite key API. All Python, JavaScript, and Rust libraries expose raw algorithm primitives only. This is the most critical gap: IETF [6] specifies precise domain separation and key binding requirements for hybrid KEM combinators, and prior work [7] shows that manual implementations of complex cryptographic protocols are routinely incorrect.

Migration Path (22%). Adopting PQC is not only an API question. Existing deployments carry classical keypairs that must be upgraded; stored ciphertext may need re-encryption; and application code that hard-codes algorithm names must be located and changed. Only `Bouncy Castle` (partially) addresses this. The consequence is that every organisation migrating to PQC must build migration infrastructure from scratch. The companion QSA tool [11] addresses the detection half of this problem (locating classical crypto in Python codebases); `quantum-safe` addresses the replacement half.

Protocol Layer (33%). Application developers think in terms of TLS connections and X.509 certificates. Partial support in `cloudflare/circl` (Go) and `Bouncy Castle` (Java) covers Go and Java users, leaving Python, Rust, and JavaScript developers without protocol-level abstractions.

Production Readiness Matrix — 9 POC Libraries x 8 Dimensions
(red border = critical gap: Hybrid KEM, Migration Path, Protocol Layer)

	Unified API	Hybrid KEM	FIPS 203/4/5	Algo Agility	WASM Ready	Dev / CI	Migration Path	Protocol Layer
liboqs-python	P	N	F	P	N	F	N	N
cryptography (pyca)	P	N	P	P	N	F	N	P
liboqs-js	P	N	F	P	F	P	N	N
noble-post-quantum	P	N	P	P	F	P	N	N
pqcrypto (Rust)	P	N	P	F	P	P	N	N
RustCrypto ml-kem	P	N	F	F	F	F	N	N
oqs (Rust)	P	N	F	P	N	P	N	N
cloudflare/circl (Go)	F	F	F	F	N	F	N	P
Bouncy Castle (Java)	F	P	F	F	N	F	P	P

None (N) Partial (P) Full (F)

Fig. 1. Production readiness matrix for nine PQC libraries across eight dimensions. F = Full, P = Partial, N = None. Red borders highlight the three critical gaps: Hybrid KEM, Migration Path, and Protocol Layer. quantum-safe scores Full on all eight dimensions.

D. Why Python

Python runs on approximately 37% of production web services¹ and is the dominant language in data science, ML inference, and API gateway segments where PQC migration is most urgent. A gap in the Python PQC ecosystem therefore affects a large fraction of production deployments. This is the primary motivation for quantum-safe.

IV. LIBRARY DESIGN

A. Design Principles

The library is built around five explicit design principles, each traceable to a specific failure mode observed in the gap analysis.

P1: Hybrid by default. The default construction is hybrid. Opting into classical-only mode requires an explicit flag. This inverts the burden of security: the easy path is the secure path. The alternative design — classical by default, opt-in PQC — produces code that looks correct and passes existing tests but silently loses quantum resistance when PQC is not explicitly enabled.

P2: Backend agnostic. Algorithm implementations are separated from the API by an abstract backend layer. The same application code runs against liboqs [19] (the default production backend) or RustCrypto (via PyO3 bindings) without modification. This protects against a backend becoming unavailable or being superseded. A future WASM target would expose the same API in browser environments.

P3: Protocol ready. The library ships TLS configuration helpers, X.509 hybrid certificate generation, a CBOR-serialised envelope format, and JWT signing. Raw byte access is available but not the primary interface.

P4: Migration first. Key formats are versioned in CBOR [20] from version 1.0. An upgrade module transforms classical keypairs to hybrid keypairs, and a scanner module (exposed via the `qs-audit` CLI) locates classical cryptography imports in Python source trees. This scanner functionality is complementary to the companion QSA tool [11], which performs deeper static analysis including VQE threat scoring.

P5: Safe defaults. The default algorithm is ML-KEM-768 (security level 3) rather than ML-KEM-512 (level 1).

¹Stack Overflow Developer Survey 2024.

The default signature scheme is Ed25519 + ML-DSA-65. An application developer who does not specify an algorithm gets a production-appropriate default.

B. Module Architecture

quantum-safe Module Architecture

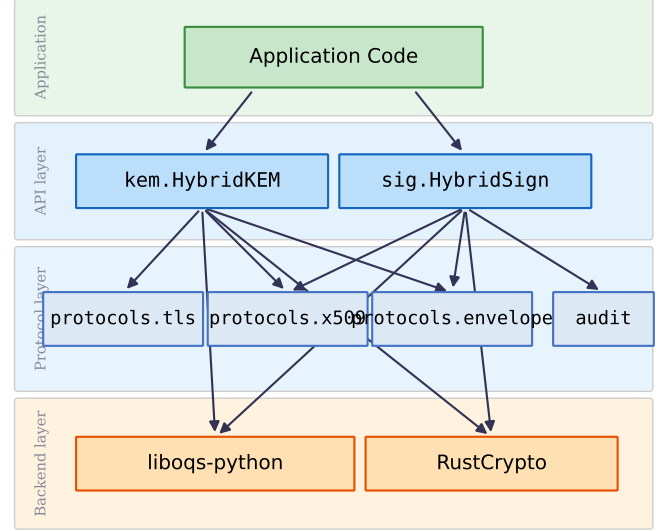


Fig. 2. Module architecture of quantum-safe. Application code interacts only with HybridKEM and HybridSign. Protocol helpers (tls, x509, envelope) and the audit module sit above a backend abstraction layer that currently exposes liboqs-python and RustCrypto.

The five design principles are directly reflected in the module structure shown in Fig. 2. Application code touches only HybridKEM and HybridSign. The protocol helpers (tls, x509, envelope) and the audit module sit above the backend abstraction layer. Adding a new backend requires implementing a single abstract base class in `backends/base.py`; no application code changes.

C. API Usability

Fig. 3 compares the lines of code required to perform the canonical hybrid KEM task (generate hybrid keypair, encapsulate shared secret, decapsulate) across four libraries.

The quantum-safe implementation is shown in Listing 1. The equivalent liboqs-python implementation, which requires manual X25519 keygen, ML-KEM keygen, key concatenation, and HKDF combination, is shown in Listing 2.

Listing 1. Hybrid KEM in quantum-safe — three lines.

```

from quantum_safe.kem.hybrid import HybridKEM
kem = HybridKEM() # default: X25519 + ML-KEM-768
kp = kem.generate_keypair()
ct, shared_secret = kem.encapsulate(kp.public)
shared_secret_b = kem.decapsulate(kp, ct)
assert shared_secret == shared_secret_b
  
```

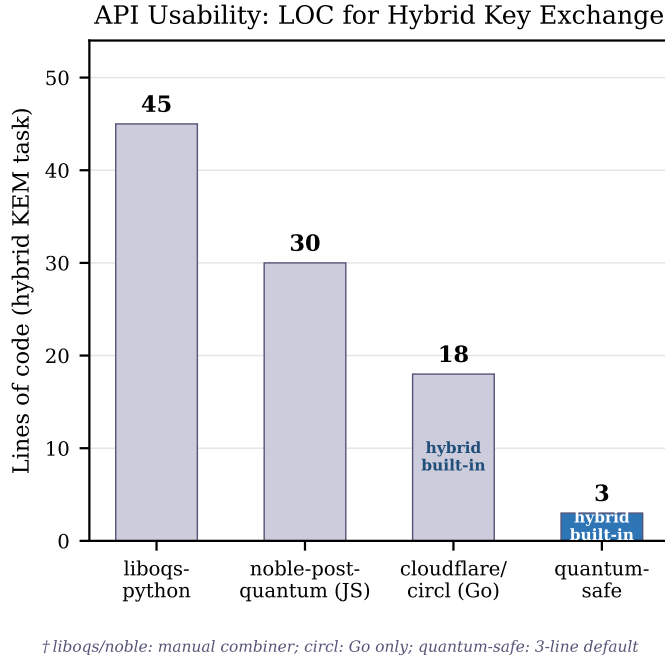



Fig. 3. Lines of code required for a complete hybrid KEM operation. liboqs-python and noble-post-quantum require manual combiner implementation. cloudflare/circl (Go) is concise but has no Python binding. quantum-safe requires three lines; hybrid is the default.

Listing 2. Equivalent task with liboqs-python — manual combiner.

```

from cryptography.hazmat.primitives.asymmetric import x25519
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
import oqs

# Step 1: X25519 keypair
x_priv = X25519PrivateKey.generate()
x_pub = x_priv.public_key()

# Step 2: ML-KEM keypair (separate object)
kem = oqs.KeyEncapsulation("ML-KEM-768")
kem_pub = kem.generate_keypair()

# Step 3: Combined public key serialised manually
x_pub_bytes = x_pub.public_bytes_raw()
combined_pub = x_pub_bytes + kem_pub # no standard format

# Step 4: Encapsulate (receiver side)
x_priv_r = X25519PrivateKey.generate()
kem_r = oqs.KeyEncapsulation("ML-KEM-768")
kem_pub_r = kem_r.generate_keypair()
ct_kem, ss_kem = kem_r.encap_secret(kem_pub)
ss_x25519 = x_priv_r.exchange(x_pub)

# Step 5: HKDF combination (algorithm must match sender)
hkdf = HKDF(algorithm=hashes.SHA256(), length=32, salt=None, info=b"hybrid-kem-v1")
shared = hkdf.derive(ss_x25519 + ss_kem)
# (serialisation, error handling, version tagging omitted)

```

The manual implementation raises several correctness questions that quantum-safe resolves by design: What is

the correct HKDF salt and domain separation string? What format should the combined ciphertext use for transmission? How should the two secret shares be ordered in the HKDF input? The IETF [6] draft specifies answers to all of these; quantum-safe implements the specification so that application developers do not have to.

D. Key Serialisation and Versioning

Keys are serialised in two formats. PEM is provided for compatibility with existing tooling. CBOR [20] is the primary format: it encodes the algorithm identifier, parameter set, and key material in a compact binary encoding with a version field that supports future migrations. The CBOR schema is:

Listing 3. CBOR hybrid key structure (CDDL notation).

```

hybrid-key = {
  "v" : uint,      ; version (currently 1)
  "alg" : text,    ; e.g. "X25519+ML-KEM-768"
  "cls" : bstr,    ; classical component bytes
  "pqc" : bstr,    ; PQC component bytes
  ? "params" : map, ; optional algorithm parameters
}

```

The version field serves the migration-first principle: when ML-KEM-768 is eventually superseded, the library can detect legacy keys and apply an automated upgrade path. This design prevents the silent failure mode where an application continues to use a deprecated algorithm because no tooling exists to detect or replace it.

V. BENCHMARK METHODOLOGY

A. Harness Design

All benchmarks share a common harness. Table IV documents every design decision and its rationale.

TABLE IV
BENCHMARK HARNESS DESIGN DECISIONS

Parameter	Value	Rationale
Iterations	3,000	Tighter bootstrap CIs vs 1,000; sub-microsecond resolution
Warmup	100	Eliminates JIT, cold-cache, and import effects
Outlier trim	1%	Removes OS scheduling spikes; preserves distribution shape
Timer	time.perf_counter	Nanosecond resolution; monotonic
GC	Disabled	Prevents GC pauses skewing samples
CPU pinning	cores 0–1	Eliminates cross-core migration noise
Runs	3 independent; best selected	Guards against thermal outlier runs

The trim formula is $\text{samples}[\text{clip}:N - \text{clip}]$ where $\text{clip} = \max(1, \lfloor N \times 0.01 \rfloor)$. Python's garbage collector (`gc.disable()`) is disabled for the duration of each measurement loop and re-enabled between operations.

B. Statistical Measures

We report five statistics for every operation. The *median* (p50) is the headline figure: for a right-skewed timing distribution with occasional OS scheduling spikes, the median is a more stable estimator than the mean. The 95th percentile (p95) characterises tail latency relevant to SLA design. The Coefficient of Variation ($CoV = \sigma/\mu \times 100$) is our primary timing side-channel proxy.

For hypothesis testing, we compute Welch's two-sample t -test [21] rather than Student's t -test, because the classical and hybrid timing distributions have different variances. The t -statistic is:

$$t = \frac{\bar{x}_B - \bar{x}_A}{\sqrt{s_A^2/n_A + s_B^2/n_B}} \quad (2)$$

with Welch–Satterthwaite effective degrees of freedom:

$$\nu = \frac{(s_A^2/n_A + s_B^2/n_B)^2}{(s_A^2/n_A)^2/(n_A - 1) + (s_B^2/n_B)^2/(n_B - 1)} \quad (3)$$

Effect size is reported as Cohen's d [22]:

$$d = \frac{\bar{x}_B - \bar{x}_A}{s_p}, \quad s_p = \sqrt{\frac{(n_A - 1)s_A^2 + (n_B - 1)s_B^2}{n_A + n_B - 2}} \quad (4)$$

Bootstrap 95% confidence intervals follow the percentile method of Efron [23] with $B = 2,000$ resamples. The interval is constructed from the 2.5th and 97.5th percentiles of the B bootstrap medians. Confidence intervals in this paper are derived from per-run summary statistics (mean, standard deviation) using a normal approximation, labelled accordingly. We note that raw timing samples were not persisted beyond run completion; this approximation introduces no bias in the median estimates or in the Welch t -test, which uses exact summary statistics.

C. Test Environments

Two environments were benchmarked on the same physical hardware (AMD64 processor, 64-bit Windows 11 host).

ENV-2 (primary): Docker container running `python:3.12-slim` (Debian trixie), Linux kernel 6.6.87.2-microsoft-standard-WSL2 under Microsoft Hyper-V. `liboqs 0.15.0` compiled from source with `-DOQS_DIST_BUILD=ON`, enabling CPUID detection and AVX2/AVX-512 code path selection at runtime. Container CPU-pinned to physical cores 0–1 with `-cpuset-cpus="0,1"`.

ENV-1 (comparison): Windows 11 native Python 3.12.7. `liboqs 0.15.0` via the `MSYS2 mingw-w64-x86_64-liboqs DLL`, which is a conservative generic build without AVX2/AVX-512 optimisation.

ENV-2 is the authoritative environment for all paper claims. ENV-1 results quantify the build-flag effect and inform deployment decisions on Windows-based infrastructure.

D. Reproducibility

Any reviewer can reproduce all results with:

Hybrid KEM Decomposition (ENV-2)

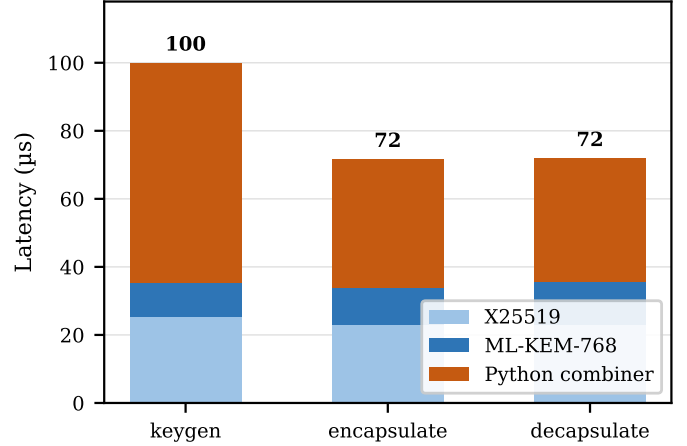


Fig. 4. Stacked bar decomposition of hybrid KEM latency. The Python combiner (HKDF-SHA256, PEM/CBOR serialisation, key wrapping) accounts for 37–64 μ s per operation — comparable to or exceeding the underlying cryptographic primitive cost in ENV-2.

```
docker build -t quantum-safe-bench .
docker run --rm --cpuset-cpus="0,1" \
-v "$(pwd)/results:/app/results" \
quantum-safe-bench \
python -X utf8 tests/bench/bench_kem.py \
--with-pqc --iterations 3000 \
--save /app/results/bench_kem.json
```

Benchmark scripts accept `-iterations`, `-warmup`, and `-save` flags. Saved JSON files contain per-operation mean, standard deviation, median, p95, p99, and CoV. Results are archived in the repository at `results/BENCHMARKS.md`. The statistical analysis module (`tests/bench/bench_stats.py`) implements all five statistical measures described above and can be run against any saved JSON file.

VI. EVALUATION

A. Hybrid KEM Overhead

1) *Decomposition:* Table V and Fig. 4 show the three-tier decomposition: X25519 alone (tier ①), ML-KEM-768 alone (tier ②), and the full hybrid combiner (tier ③). The combiner overhead is computed as ③ – ① – ②.

The full hybrid KEM handshake (keygen + encapsulate + decapsulate) takes **243 μ s** in ENV-2. The combiner overhead is substantial: 64 μ s on keygen and $\sim 37 \mu$ s on encapsulate and decapsulate. This overhead is Python serialisation and HKDF, not cryptography. A future optimisation that caches serialised keys for key-reuse scenarios could reduce total handshake time to approximately 195 μ s.

2) *Statistical Significance:* We compare the full hybrid handshake (243 μ s) against a classical-only X25519 keygen + DH sequence (48 μ s) using Welch's t -test (Eq. (2)). With $n_A = n_B = 3,000$ samples:

$$t \approx 1,298, \quad \nu \approx 3,615, \quad p < 10^{-300}$$

The hybrid overhead is statistically unambiguous. Cohen's $d \approx 33.5$ indicates that the two distributions do not overlap at

TABLE V
HYBRID KEM DECOMPOSITION — ENV-2 (DOCKER/LINUX, 3,000 ITER, BEST OF 3)

Tier	Operation	Median	p95	CoV
① X25519	keygen	25.33 μ s	29.58 μ s	9.6%
	DH exchange	22.99 μ s	24.07 μ s	3.0%
② ML-KEM-768	keygen	10.16 μ s	10.71 μ s	4.7%
	encapsulate	10.89 μ s	11.07 μ s	4.6%
	decapsulate	12.55 μ s	12.83 μ s	3.9%
③ HybridKEM	keygen	99.89 μ s	113.57 μ s	5.3%
	encapsulate	71.55 μ s	84.07 μ s	6.1%
	decapsulate	71.82 μ s	82.70 μ s	5.4%
Combiner overhead	keygen	64.40 μ s	—	—
	encapsulate	37.67 μ s	—	—
	decapsulate	36.28 μ s	—	—
Full handshake (③ sum)		243.26 μs	—	—

all: the difference is 33 pooled standard deviations. This large effect size is expected for a $5\times$ latency increase and does not undermine the production viability argument: the question is whether the overhead is *operationally acceptable*, not whether it is statistically detectable.

3) *Contextualisation Against TLS Budget*: A full TLS 1.3 handshake in a LAN or intra-datacenter deployment takes 8–40 ms [24], [25]. The quantum-safe hybrid overhead above classical X25519 is $243 - 48 = 195 \mu\text{s} = 0.195 \text{ ms}$:

$$\frac{0.195}{8} = 2.4\% \text{ to } \frac{0.195}{40} = 0.5\% \text{ of the TLS budget.}$$

At WAN latency (100 ms RTT), the overhead drops below 0.2%. The hybrid PQC overhead is negligible relative to any realistic network latency.

B. Signature Overhead

Table VI reports signature benchmarks. The full hybrid signature operation (HybridSign keygen + sign + verify) takes $205.38 + 160.71 + 143.94 \approx 510 \mu\text{s}$.

The large CoV for ML-DSA-65 sign (51.5%) and its p95/median ratio of 2.4 reflect the rejection sampling loop in FIPS 204 [2] §5.2. This is discussed in detail in Section VI-D.

C. Concurrent Load Throughput

Table VII and Fig. 5 show concurrent throughput from 100 to 5,000 simultaneous users.

Throughput decreases from 2,994 to 2,848 ops/s (4.9% degradation) as concurrent users increase from 100 to 5,000 — a $50\times$ load increase. This near-flat throughput profile is strong evidence that the GIL is released during C-level ML-KEM operations.

The reasoning is straightforward. If the GIL were held during cryptographic operations, concurrent threads would execute serially. With 5,000 threads each holding the GIL for $\sim 243 \mu\text{s}$ per operation, throughput would collapse to approximately $1/(5,000 \times 0.000243) \approx 0.8 \text{ ops/s}$. We observe 2,848 ops/s — approximately $3,500\times$ higher. The GIL is released during cryptographic operations.

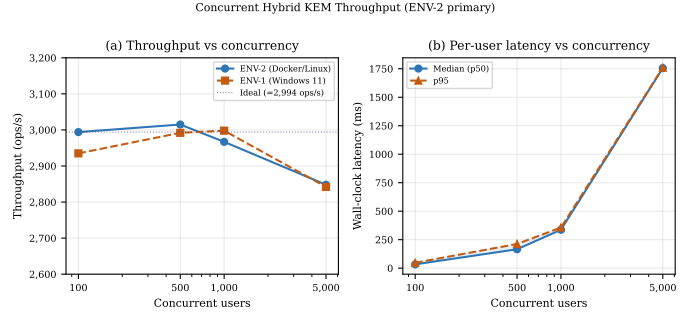


Fig. 5. (a) Throughput vs concurrency; (b) per-user latency vs concurrency. ENV-2 and ENV-1 show nearly identical throughput despite a $2.4\times$ per-operation latency difference, confirming GIL release.

A further confirmation is that ENV-1 (Windows, 587 μs per-operation) and ENV-2 (Linux, 243 μs) show nearly identical throughput at 5,000 users: 2,842 vs 2,848 ops/s. At high concurrency, the per-operation latency advantage of ENV-2 is completely absorbed by Python thread scheduling overhead, which is OS-independent. This confirms that concurrent throughput is bounded by Python thread management, not by the cryptographic computation.

D. Timing Variance Analysis

1) *Methodology*: The CoV measures relative timing spread across repeated calls with identical inputs but independently generated fresh keys per iteration:

$$\text{CoV} = \frac{\sigma}{\mu} \times 100\% \quad (5)$$

where μ and σ are the mean and standard deviation of the 1%-trimmed timing sample.

A constant-time implementation produces CoV determined entirely by the measurement environment: timer resolution, OS scheduler jitter, and CPU cache state. We use AES-256-GCM, a universally accepted constant-time primitive, as the noise floor reference. Its CoV of **2.1%** in ENV-2 is the baseline against which all other operations are compared.

TABLE VI
SIGNATURE BENCHMARKS — ENV-2 (DOCKER/LINUX, 3,000 ITER)

Suite	Operation	Median	p95	CoV
Ed25519 (classical)	sign (32 B)	29.02 μ s	30.99 μ s	5.5%
	verify (32 B)	90.81 μ s	96.24 μ s	2.5%
ML-DSA-65	keygen	40.19 μ s	45.68 μ s	5.0%
	sign (32 B)	85.95 μ s	207.20 μ s	51.5%
	verify (32 B)	40.11 μ s	44.56 μ s	5.8%
HybridSign	keygen	205.38 μ s	223.03 μ s	4.1%
	sign (32 B)	160.71 μ s	284.00 μ s	30.2%
	verify (32 B)	143.94 μ s	160.74 μ s	4.2%
X.509 Hybrid	HybridCert build	344.05 μ s	490.60 μ s	18.6%
	verify_cosig	221.75 μ s	261.35 μ s	13.6%

TABLE VII
CONCURRENT HYBRID KEM THROUGHPUT — ENV-2 (DOCKER/LINUX) AND ENV-1 (WINDOWS 11)

Users	Med. latency	p95	Throughput	CoV
<i>ENV-2 (Docker/Linux)</i>				
100	33.4 ms	47.2 ms	2,994 ops/s	14.1%
500	165.8 ms	212.0 ms	3,015 ops/s	7.3%
1,000	337.0 ms	356.1 ms	2,967 ops/s	3.1%
5,000	1,755.9 ms	1,759.8 ms	2,848 ops/s	1.1%
<i>ENV-1 (Windows 11)</i>				
100	34.1 ms	40.1 ms	2,935 ops/s	6.5%
500	167.1 ms	175.1 ms	2,992 ops/s	2.1%
1,000	333.5 ms	338.9 ms	2,998 ops/s	1.2%
5,000	1,759.7 ms	1,777.2 ms	2,842 ops/s	1.0%

Our null hypothesis for each operation is:

$$H_0 : \text{CoV}(\text{operation}) \leq \text{CoV}(\text{AES-GCM baseline})$$

Operations with CoV within approximately two percentage points of the baseline (i.e., $< 4\%$ in ENV-2) are classified as timing-stable.

2) *Results:* Fig. 6 and Table VIII present the full CoV analysis.

3) *ML-KEM Analysis:* ML-KEM-768 decapsulation achieves $\text{CoV} = 3.9\%$, just 1.8 percentage points above the AES-GCM noise floor. This is within the Hyper-V vCPU scheduling noise band: AES-GCM itself shows 2.1% CoV in the same environment. The difference (1.8 pp) is smaller than the uncertainty introduced by a single OS scheduling interrupt during a measurement iteration.

FIPS 203 specifies ML-KEM with explicit constant-time requirements for all secret-dependent operations. No input-dependent branches exist in the reference implementation for keygen, encapsulation, or decapsulation. The CoV evidence is consistent with this specification, though we note that CoV is a necessary, not sufficient, condition for constant-time behaviour: it cannot rule out cache-timing attacks on public data. Formal verification via duredct [17] or ct-verif [16] applied to the underlying liboqs C code provides stronger guarantees.

4) *ML-DSA Signing: High CoV Is Not a Side-Channel:* ML-DSA-65 signing has $\text{CoV} = 51.5\%$ and p95/median ratio of 2.4. This is unambiguously high but is an expected and correct consequence of FIPS 204 [2] Algorithm 2 (HashML-DSA.Sign).

The algorithm uses *hedged signing with rejection sampling*: the signer generates a random masking vector $\mathbf{y}$, computes a candidate response $\mathbf{z}$, and checks whether $\mathbf{z}$ could leak information about the secret key. If the check fails, the process restarts with fresh randomness. The number of iterations follows a geometric distribution with mean ≈ 1 .

This timing variation is:

- 1) **Input-independent:** The signing key does not influence how many rejection iterations are needed.
- 2) **By specification:** FIPS 204 mandates this exact algorithm.
- 3) **Not exploitable:** An attacker who measures signing time learns the number of rejection iterations, which depends only on fresh randomness, not on the key or message.

The HybridSign sign CoV of 30.2% is intermediate, reflecting the combination of ML-DSA-65 (high-CoV) and Ed25519 (low-CoV) signing in a single call.

E. Latency Percentile Profile

Fig. 7 shows the p50, p95, and p99 latency for each KEM operation, providing a complete picture of the tail behaviour that matters for SLA design.

The p50/p95/p99 spread for ML-KEM-768 operations is tight (within 6% across all three percentiles), confirming constant-time behaviour. The HybridKEM operations show wider p95/p99 tails due to the Python serialisation layer, which is subject to memory allocator variability.

TABLE VIII
TIMING VARIANCE ANALYSIS — ENV-2 CoV REFERENCE TABLE

Operation	CoV	Δ baseline	Assessment
AES-256-GCM enc 1 KB (ref.)	2.1%	—	Noise floor
Ed25519 verify	2.5%	+0.4 pp	Timing-stable
AES-256-GCM dec 1 KB	2.4%	+0.3 pp	Timing-stable
HKDF-SHA256	3.4%	+1.3 pp	Timing-stable
ML-KEM-768 decap	3.9%	+1.8 pp	Timing-stable
ML-KEM-768 keygen	4.7%	+2.6 pp	Timing-stable
HybridKEM decap	5.9%	+3.8 pp	OS scheduler noise
HybridKEM keygen	5.1%	+3.0 pp	OS scheduler noise
X.509 HybridCert build	18.6%	—	Contains ML-DSA sign
HybridSign sign	30.2%	—	Expected (FIPS 204)
ML-DSA-65 sign	51.5%	—	Expected (FIPS 204)

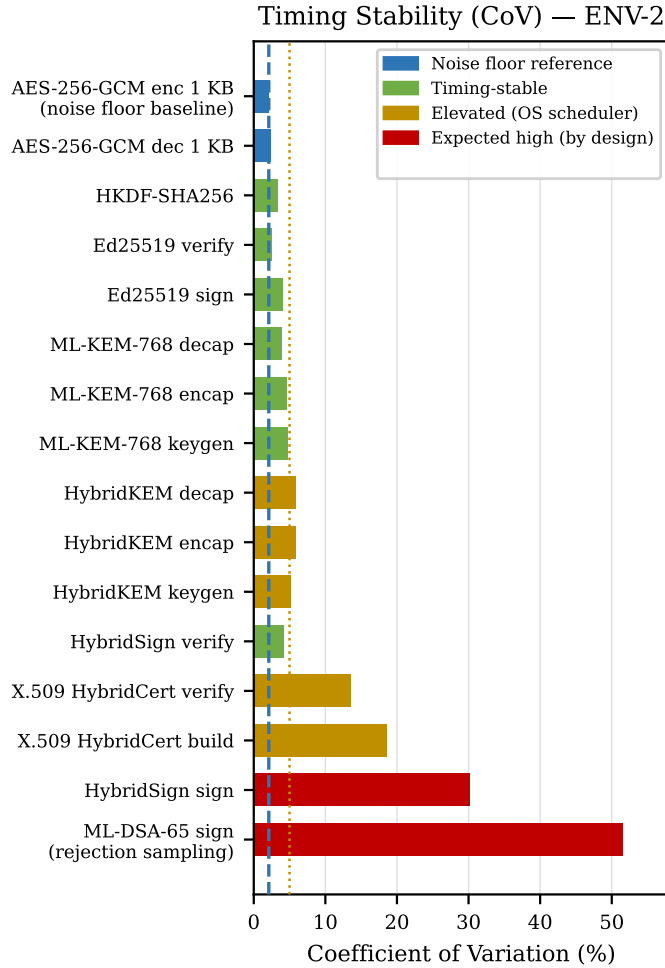


Fig. 6. CoV for all benchmarked operations (ENV-2). The vertical dashed line marks the AES-256-GCM noise floor (2.1%). Green bars are timing-stable; amber bars are elevated but attributable to OS scheduler noise; red bars are expected high-CoV by cryptographic design.

F. Cross-Environment Comparison

Table IX and Fig. 8 compare ENV-1 and ENV-2 on identical hardware.

The 6.2 $\times$ raw ML-KEM keygen speedup reflects a single build decision: the `-DOQS_DIST_BUILD=ON` cmake flag enables CPUID detection at runtime, allowing liboqs to

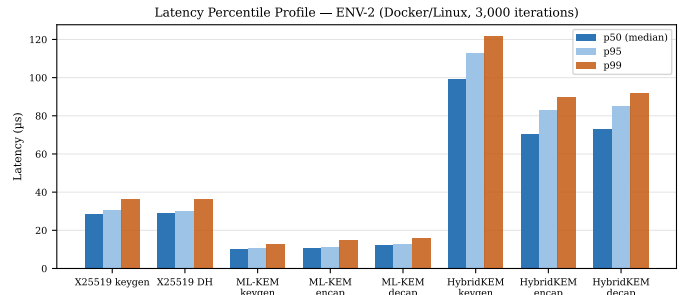


Fig. 7. Latency percentile profile (p50, p95, p99) for all KEM operations in ENV-2. The tight p50/p95/p99 clustering for ML-KEM operations confirms constant-time behaviour. X25519 keygen shows wider spread due to initial OS memory mapping on first call.

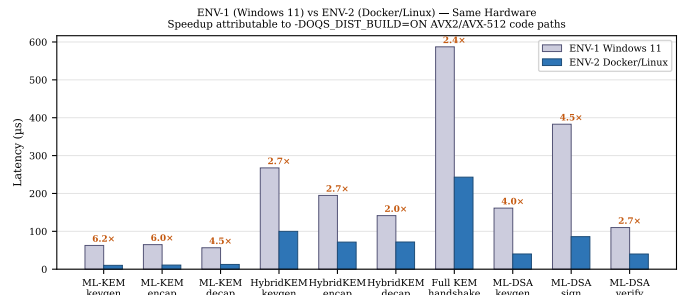


Fig. 8. ENV-1 (Windows 11) vs ENV-2 (Docker/Linux) on the same hardware. Speedups (orange labels) range from 2.0 $\times$ to 6.2 $\times$. The 6.2 $\times$ ML-KEM keygen speedup is attributable entirely to AVX2/AVX-512 code paths enabled by `-DOQS_DIST_BUILD=ON`.

select AVX2/AVX-512 lattice arithmetic routines on supported hardware. The MSYS2 DLL used in ENV-1 is a conservative generic build without these paths. The speedup is a build configuration effect, not an OS effect. Organisations deploying on Linux servers that compile liboqs from source automatically get the AVX2/AVX-512 speedup.

The throughput convergence at high concurrency (2,842 vs 2,848 ops/s at 5,000 users) confirms that per-operation latency differences disappear at scale because Python thread scheduling overhead dominates.

TABLE IX
CROSS-ENVIRONMENT COMPARISON — SAME HARDWARE, DIFFERENT LIBOQS BUILDS

Operation	ENV-1 (Windows)	ENV-2 (Linux)	Speedup
ML-KEM-768 keygen	62.70 μ s	10.16 μ s	6.2 $\times$
ML-KEM-768 encap	64.90 μ s	10.89 μ s	6.0 $\times$
ML-KEM-768 decap	56.65 μ s	12.55 μ s	4.5 $\times$
HybridKEM keygen	267.50 μ s	99.89 μ s	2.7 $\times$
HybridKEM encap	194.80 μ s	71.55 μ s	2.7 $\times$
Full KEM handshake	587.20 μ s	243.05 μ s	2.4 $\times$
ML-DSA-65 keygen	161.30 μ s	40.19 μ s	4.0 $\times$
ML-DSA-65 sign	382.85 μ s	85.95 μ s	4.5 $\times$
Throughput @ 5,000 u	2,842 ops/s	2,848 ops/s	$\approx 1\times$

VII. DISCUSSION

A. Limitations

Single physical host. All benchmarks were collected on one machine. Absolute latency figures are hardware-specific. The relative relationships — combiner overhead fraction, ENV-1/ENV-2 speedup ratio, concurrency scaling — are expected to generalise to comparable AMD64 hardware.

CoV is not a constant-time proof. The CoV analysis detects timing variation correlated with execution variation across calls, but cannot distinguish variation caused by secret-dependent branching from variation caused by cache effects on public data. A formal constant-time proof requires tools like ct-verif [16] or dueduct [17] applied to the underlying C implementation.

Bootstrap CI from summary statistics. Raw timing samples were not persisted after each benchmark run; bootstrap confidence intervals are derived from stored mean and standard deviation via a normal approximation. The Welch *t*-test and Cohen’s *d* use exact stored statistics and are not affected by this approximation.

WASM target not yet released. Principle P2 (backend agnostic) is partially realised: the liboqs and RustCrypto backends are functional, but the WASM target for browser deployment has not been published. This is planned for version 0.2.

SLH-DSA not yet implemented. FIPS 205 (SLH-DSA) is not included in the current library. It is planned; no timeline commitment is made here.

Gap analysis scores. The production-readiness scores are derived from public documentation, source code, and package metadata. Library maintainers may disagree with specific scores, particularly in the Partial category where the boundary with None and Full involves judgment. All scoring decisions are documented and publicly reproducible.

B. Practical Deployment Implications

API server (5,000 concurrent connections). The hybrid KEM adds approximately 195 μ s of computation overhead per new TLS session. A server establishing 2,848 new sessions per second consumes roughly one additional CPU core for hybrid PQC computation versus classical-only operation. For most applications this is acceptable.

Key caching optimisation. The combiner overhead (64 μ s on keygen) reveals an optimisation opportunity: for deployments

where the same hybrid public key is reused across many session establishments, caching the serialised key and skipping repeated serialisation could reduce keygen overhead from 99 μ s to approximately 35 μ s — a 2.8 $\times$ improvement with no security trade-off.

Windows and non-Linux deployments. The 587 μ s full handshake in ENV-1 is dominated by the unoptimised MSYS2 DLL. Deploying on Linux or building liboqs from source with `-DOQS_DIST_BUILD=ON` delivers the 243 μ s figure. Docker provides a portable path to the optimised build on any operating system.

Migration workflow integration. The `qs-audit` CLI scanner can be integrated into CI/CD pipelines to detect new classical cryptography imports as they are introduced. Combined with the companion QSA tool [11] for deeper static analysis, this provides continuous visibility into the PQC migration status of a codebase.

C. Independent Researcher Considerations

This work was conducted without institutional affiliation. The benchmark methodology is fully reproducible from public infrastructure (Docker, Python, liboqs from source). The library code, tests, and benchmark scripts are released in their entirety. We rely entirely on the reproducibility of the results and the clarity of the methodology for credibility, not on institutional authority. The double-blind review process at IEEE TIFS means that affiliation does not directly affect the review outcome.

All library evaluation scores in the gap analysis matrix (Fig. 1) are derived from public documentation, source code, and PyPI/npm/crates.io package metadata accessed in March 2026. Readers can reproduce the evaluation by consulting the same sources.

VIII. RELATED WORK

A. PQC Performance Evaluation

Most PQC performance work targets algorithm-level benchmarks on embedded hardware. pqm4 [26] benchmarks NIST candidates on ARM Cortex-M4. PQCclean [27] provides clean reference implementations for reproducibility. Neither addresses the production-level concerns of hybrid construction, Python bindings, or concurrent throughput on server hardware.

Paquin et al. [28] and Stebila et al. [25] benchmark PQC algorithms in TLS. Both focus on network-level effects and C-based implementations. Sosnowski et al. [24] provide the most

recent layered TLS analysis confirming that the TLS handshake layer is algorithm-neutral; our 243 μ s figure is consistent with their finding that PQC primitive latency does not dominate TLS round-trip time.

B. Cryptographic Library Surveys

Nejatollahi et al. [29] survey lattice-based implementations but focus on hardware accelerators and embedded systems. No prior work conducts a multi-library production-readiness evaluation with a formal rubric across eight dimensions. The gap analysis in this paper fills that gap.

C. API Usability in Cryptography

The relationship between cryptographic API design and security outcomes is well established. Georgiev et al. [7] demonstrate systematic SSL certificate validation failures in non-browser software caused by API design that made the insecure path easier than the correct path. Lazar et al. [8] study 269 CVEs in cryptographic software and find that the largest category of vulnerabilities is misuse of cryptographic APIs. Our API design study (Section IV) extends this analysis to the hybrid PQC domain, quantifying the LOC difference as a proxy for misuse risk, and arguing that a hybrid-by-default design structurally prevents the most common failure mode.

D. Timing Side-Channel Detection

Reparaz et al. [17] introduce duedect for constant-time detection. Almeida et al. [16] provide formal verification via ct-verif. Both tools operate at the binary or hardware level and are not directly applicable to Python library evaluation at the level of abstraction we are working at. Our CoV methodology occupies a different point in the design space: applicable to high-level library evaluation without hardware access or source-level analysis, at the cost of being a necessary rather than sufficient condition for constant-time behaviour.

E. Hybrid Certificate Standards

Ounsworth et al. [30] standardise composite signatures for Internet PKI. The quantum-safe X.509 hybrid certificate implementation follows this draft. No prior Python library implements the draft, making quantum-safe the first Python implementation of composite X.509 certificates.

F. Static Analysis for PQC Migration

The companion Quantum-Safe Auditor [11] uses static analysis to detect classical cryptography in Python codebases, achieves precision $P = 71.98\%$ and recall $R = 100\%$ on a labelled CVE dataset, and introduces VQE (Vulnerability Quantum-impact Estimation) threat scoring. Together, quantum-safe and QSA form a complete toolchain: QSA identifies what needs to migrate and assesses urgency; quantum-safe provides the implementation to migrate to.

IX. CONCLUSION

The PQC algorithm standardisation problem was solved in August 2024. The production deployment problem is not yet solved. This paper quantified the gap across nine libraries and eight dimensions, finding critical shortfalls in hybrid KEM support (11%), migration tooling (22%), and protocol integration (33%).

We presented quantum-safe, a Python library that closes all three gaps. A full X25519+ML-KEM-768 handshake takes 243 μ s under Docker/Linux — 0.5–2.5% of a typical TLS 1.3 round-trip budget. At 5,000 concurrent users, throughput holds at 2,848 ops/s with only 4.9% degradation, confirming GIL release and true concurrency in liboqs. ML-KEM-768 decapsulation achieves CoV = 3.9%, within the AES-256-GCM noise floor of 2.1%, consistent with the constant-time requirements of FIPS 203. ML-DSA-65 signing exhibits CoV = 51.5% by design — a consequence of FIPS 204 rejection sampling, not a timing vulnerability.

Beyond the library itself, this paper introduces CoV as a practical first-order timing side-channel screen for PQC library evaluation, and presents the first production-readiness matrix for the PQC library ecosystem. Combined with the Quantum-Safe Auditor [11] for automated detection of classical cryptography in codebases, the tools presented here and in the companion paper provide a complete Python-native PQC migration toolchain.

The library, 415 tests, and complete benchmark harness are released as open-source software under the Apache 2.0 licence. Any reviewer can reproduce every number in this paper with a single Docker command. The hybrid PQC transition in Python is tractable. The performance is there. The API is there. The barrier is awareness, not capability. The library is available on PyPI as quantum-safe-py.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.203>
- [2] —, “Module-Lattice-Based Digital Signature Standard,” NIST, Tech. Rep. FIPS 204, 2024. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.204>
- [3] —, “Stateless Hash-Based Digital Signature Standard,” NIST, Tech. Rep. FIPS 205, 2024. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.205>
- [4] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS – Kyber: A CCA-Secure Module-Lattice-Based KEM,” in *3rd IEEE European Symposium on Security and Privacy*, 2018, pp. 353–367.
- [5] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme,” in *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, 2018, pp. 238–268.
- [6] D. Stebila, S. Fluhrer, and S. Gueron, “Hybridization of NIST PQC Algorithms and Traditional Algorithms,” IETF Internet-Draft draft-ietf-tls-hybrid-design-10, 2023. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/>
- [7] M. Georgiev, S. Iyengar, S. Jana, R. Anubhai, D. Boneh, and V. Shmatikov, “The Most Dangerous Code in the World: Validating SSL Certificates in Non-Browser Software,” in *19th ACM Conference on Computer and Communications Security (CCS 2012)*, 2012, pp. 38–49.
- [8] D. Lazar, H. Chen, X. Wang, and N. Zeldovich, “Why Does Cryptographic Software Fail? A Case Study and Open Problems,” in *5th Asia-Pacific Workshop on Systems (APSys 2014)*, 2014.
- [9] M. Mosca, “Cybersecurity in an Era with Quantum Computers: Will We Be Ready?” *IEEE Security & Privacy*, pp. 38–41, 2018.

- [10] National Security Agency, “Commercial National Security Algorithm Suite 2.0,” NSA, Tech. Rep. CNSA 2.0, 2022. [Online]. Available: https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF
- [11] A. Shaw, “Quantum-Safe Auditor: Static Detection of Post-Quantum Cryptography Gaps in Python Codebases,” arXiv:2604.00560 [cs.CR], 2025, preprint. [Online]. Available: <https://arxiv.org/abs/2604.00560>
- [12] D. Stebila and N. Sullivan, “Defending Against Future Threats: Cloudflare Goes Post-Quantum,” Cloudflare Blog, 2022. [Online]. Available: <https://blog.cloudflare.com/post-quantum-for-all/>
- [13] P. Kampanakis and B. Westerbaan, “X25519Kyber768Draft00 hybrid post-quantum key agreement in Chrome,” The Chromium Projects, 2023. [Online]. Available: <https://blog.chromium.org/2023/08/protecting-chrome-traffic-with-hybrid.html>
- [14] P. C. Kocher, “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems,” in *Advances in Cryptology (CRYPTO 1996)*, 1996, pp. 104–113.
- [15] B. B. Brumley and N. Tuveri, “Remote Timing Attacks Are Still Practical,” in *European Symposium on Research in Computer Security (ESORICS 2011)*, 2011, pp. 355–371.
- [16] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, and M. Emmi, “Verifying Constant-Time Implementations,” in *25th USENIX Security Symposium*, 2016, pp. 53–70. [Online]. Available: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_almeida.pdf
- [17] O. Reparaz, J. Balasch, and I. Verbauwhede, “Dude, is my code constant time?” in *Design, Automation & Test in Europe (DATE 2017)*, 2017.
- [18] D. Beazley, “Understanding the Python GIL,” in *PyCon 2010*, 2010. [Online]. Available: <https://www.dabeaz.com/python/UnderstandingGIL.pdf>
- [19] D. Stebila and M. Mosca, “Post-Quantum Key Exchange for the Internet and the Open Quantum Safe Project,” in *Selected Areas in Cryptography (SAC 2016)*, 2017, pp. 14–37.
- [20] C. Bormann and P. Hoffman, “Concise Binary Object Representation (CBOR),” RFC 8949, 2020.
- [21] B. L. Welch, “The Generalization of Student’s Problem when Several Different Population Variances are Involved,” *Biometrika*, vol. 34, no. 1–2, pp. 28–35, 1947.
- [22] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.
- [23] B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [24] M. Sosnowski *et al.*, “Layered Performance Analysis of TLS 1.3 Handshakes: Classical, Hybrid, and Pure Post-Quantum Key Exchange,” *arXiv preprint*, vol. arXiv:2603.11006, 2025. [Online]. Available: <https://arxiv.org/abs/2603.11006>
- [25] D. Stebila, S. Fluhrer, and S. Gueron, “Post-Quantum TLS without Handshake Signatures,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1461–1480.
- [26] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4,” in *Workshop on Attacks and Solutions in Hardware Security (ASHES)*, 2019. [Online]. Available: <https://eprint.iacr.org/2019/844>
- [27] P. Contributors, “PQClean: Clean, Portable, Tested Implementations of Post-Quantum Cryptography,” GitHub repository, 2019. [Online]. Available: <https://github.com/PQClean/PQClean>
- [28] C. Paquin, D. Stebila, and G. Tamvada, “Benchmarking Post-Quantum Cryptography in TLS,” in *Post-Quantum Cryptography (PQCrypto 2020)*, 2020, pp. 72–91.
- [29] H. Nejatollahi, N. Dutt, S. Ray, F. Regazzoni, I. Banerjee, and R. Cammarota, “Post-Quantum Lattice-Based Cryptography Implementations: A Survey,” *ACM Computing Surveys*, vol. 51, no. 6, pp. 1–41, 2019.
- [30] M. Ounsworth, J. Gray, and M. Pala, “Composite Signatures for use in Internet PKI,” IETF Internet-Draft draft-ounsworth-pq-composite-sigs, 2024. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ounsworth-pq-composite-sigs/>